

→ Data :

Known facts that can be recorded and have an implicit meaning

Database :

Collection of related data

DBMS :

DB + Software

Mini-world :

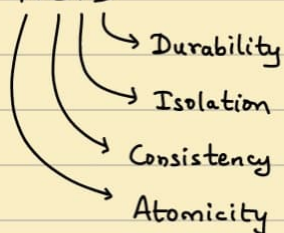
Some part of the real world about which data is stored in DB.

6/10

Triggers and Stored Procedure → Not part of syllabus

Keywords : Entity , Attribute , Relations

- ACID



- Constraints , Operations , Datatypes

Database schema → Description | Catalogue

State → Snapshot

- Three - State Architecture

Two - tier Client - Server architecture

- DDL ← Data creation

DML ← Query Language

→ Create the following for an event of Infinium :

- (a) Tables
 - (b) Columns
 - (c) Datatypes
 - (d) Relationships
- } Database Schema

Database schema :

Infinium → Mini-world

e.g. Euphony :

STUDENTS form a band with other STUDENTS.

BAND has multiple roles - Singer, Pianist, Guitarist, Drummer, etc.

BAND gives auditions and proceed further if selected.

CLUB MEMBERS judge the auditions. They are also STUDENTS.

BANDS performs SONGs.

BANDS are given positions based on their performance

Band ID	Performer ID
int	int

Performer ID	Talent
int	varchar

Band ID	Selected/NOT
int	varchar

Band ID	Song ID
int	int

Band ID	Rank
int	int

Song ID	Song name
int	varchar

1 Band → n Performers

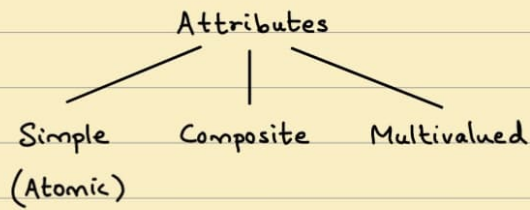
1 Band → n songs

1 Performer → 1 Talent

9/10

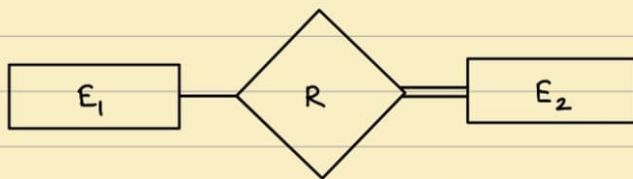
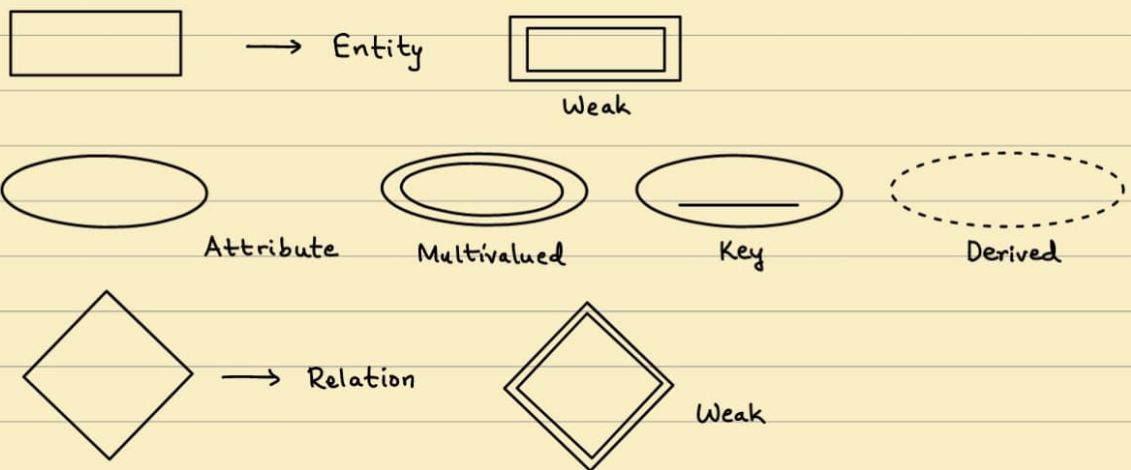
→ ER Model

- Database Designer : doesn't care how the data is captured.
 - Entity : Basic concept of ER Model (Row)
- Attributes : Properties of entity (column)



Key Attribute → Each entity has a unique value

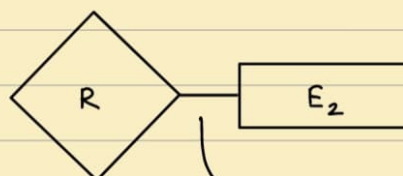
Representation : Entity. Attribute



→ Total participation of E_2 in R



→ Cardinality Ratio → 1:N for $E_1:E_2$ in R



→ (min-max) → Structural constraint

13/10

Relation types

Constraints

Recursive relations

16/10

- Weak Entity $\rightarrow$ Depends on strong Entity



- Attributes of Relationship types

- Cardinality ratio

Participation constraint

- min-max notation

If you query often $\rightarrow$ Derived Attribute X
Relationship type $\checkmark$

$\therefore$ Fetching is much faster

$\rightarrow$ Data Models:

(1) ER Models

(2) UML Class diagram

Entities $\leftrightarrow$ Classes

Relationships $\leftrightarrow$ Lines b/w classes

$\rightarrow$ DBMS:

Scripting Language

Embedded Approach

- Interface

- Utilities $\rightarrow$ Functions

- Cambridge Analytica - 3500 features stored about you

- Centralised DBMS Architecture

Client - Server Architecture $\rightarrow$ 3-Tier

→ Relational Model :

- Relation → Set of rows, i.e. tuple
Attribute → Column
→ Ordered set of values (1 tuple : 1 Row)
- Key : Uniquely identifies a row in a table
→ Can be set of attributes (or) attribute

- Schema : Table definition

$R(A_1, A_2, \dots, A_n)$
→ Attributes
→ name of the relation

- Domain :

- All possible column values
- Datatype constraint
Format constraint
- Developed from Attribute name

- Relation State :

$r(R)$ → Subset of Relation R

$$r(R) = \{t_1, t_2, \dots, t_n\}$$

$$t_i = \langle v_1, v_2, \dots, v_n \rangle$$

v_j → element of domain of A_j

$$r(R) \subset \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

The tuples are not considered to be ordered.

The attributes are ordered. ← Self-Describing

23/10

- Value : Atomic

value v_i of attribute A_i for tuple $t \leftarrow t[A_i] = v_i$

- Constraints :

- (1) Inherent (Implicit) : No duplicates
- (2) Schema-based (Explicit) : DB design
- (3) Application-based (Semantic) : By programmers

- Super Key : Attribute / set of attributes that uniquely identify all attributes in a relation
i.e. for any 2 distinct tuples t_1 & t_2 in $r(R)$: $t_1[SK] = t_2[SK]$
- Minimal Superkey $\rightarrow$ Key / Candidate Key
- Primary Key $\rightarrow$ Smallest of the candidate Keys
 $\rightarrow$ Cannot be NULL
- Foreign Key $\rightarrow$ Referential Integrity constraint

27/10

$\rightarrow$ Keys

$\rightarrow$ Integrity Violation

- Domain constraint
- Key constraint
- Referential Integrity
- Entity Integrity

RESTRICT
SET NULL
CASCADE

ATION — Owner of Schema

$\rightarrow$ Collection of Multiple Schemas

SELECT Fname, Mname, Lname

FROM EMPLOYEE

WHERE Dno = (SELECT Dnumber
FROM DEPARTMENT
WHERE Dname = "RESEARCH")

3/11

AS
LIKE
BETWEEN
ORDER BY



6/11

UPDATE SYNTAX

DELETE SYNTAX

JOINS

- Natural JOIN
- Outer JOIN (left & Right)
- Inner JOIN

10/11

→ Functional Dependencies :

- No normalization → Redundant information (Storage waste)
 ↳ INSERT, UPDATE & DELETE Anomalies

• Guidelines :

- 1) Only keep relevant information
- 2) Avoid Anomalies
- 3) Reduce NULLs
- 4) Avoid Spurious tuples on JOIN

$$(r(R) : (t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y])) \Rightarrow "X \rightarrow Y"$$

• Normalization :

1st Normal Form

2nd Normal Form

3rd Normal Form

BCNF (3.5 Normal Form) → Keys + Functional Dependencies

4th Normal Form → Keys + Multivalued Dependencies

5th Normal Form → Keys + Join Dependencies

→ Class Activity - 10/11

(I)

```
① CREATE TABLE EVENT (  
    EventID INT PRIMARY KEY,  
    Event_Name VARCHAR(20),  
    Location VARCHAR(20),  
    Time TIME,  
    Club_conducting VARCHAR(20),  
    Participant_Name VARCHAR(30),  
    WinnerID INT PRIMARY KEY,  
    Winner_Name VARCHAR(30),  
    Prize_money INT,  
    Rank INT  
);
```

```
CREATE TABLE STUDENT (  
    Roll_no INT PRIMARY KEY,  
    Name VARCHAR(30),  
    Dept VARCHAR(4),  
    Event_registered(20)
```

```
);
```

(II)

- ① INSERT INTO EVENTS (1, 'Megathon', 'H10S', 09:00:00, 'OSDQ', 'JOHN', NULL, NULL, NULL, NULL);
- ② UPDATE EVENTS
 SET Location = 'Amphi'
 WHERE Event_name = 'Megathon';
- ③ DELETE FROM EVENTS
 WHERE Club_conducting = 'Hacking Club';

(III)

Event_name → {Location, Time}
{Winner_name, Event_name} → Prize_money } may exist

Club_conducting → Event_name
Location → Time } may not exist